Space Colony – OOP Project Documentation

Team Members

Enkhjin Sanchir · Oyungerel Munkhjin · Darikhishig Batbaatar

1. Project Overview

Space Colony is an Android application developed in Java using Android Studio. The player manages a crew in a space colony by recruiting crew members, training them in a simulator, and sending them on cooperative missions against system-generated threats.

The application is built following object-oriented programming principles including encapsulation, inheritance, and polymorphism throughout all layers of the codebase.

1.1 Application Description

Players recruit crew members from five specializations — Pilot, Engineer, Medic, Scientist, and Soldier — each with unique stat values. Crew members can be moved between three locations: Quarters (home base), Simulator (training area), and Mission Control (battle preparation). Missions are turn-based and tactical, where the player chooses actions, each turn against automatically generated threats that increase in difficulty over time.

1.2 GitHub Repository

GitHub Link: SpaceColony

1.3 Video Demonstration

VideoLink tap here : SpaceColonyRecording

2. Team and Division of Work

All team members collaborated on testing, integration, and final documentation.

Team Member	What They Did
Enkhjin Sanchir	Built the main OOP structure — CrewMember base class, all 5 subclasses, Threat class, MissionAction interface and the 3 action classes (Attack, Defend, Special). Also worked on mission logic.
Oyungerel Munkhjin	Built the Android UI — all fragments, RecyclerView lists, crew card layouts, crew images, and the bottom navigation.
Darikhishig Batbaatar	Built the Statistics screen, MedBay system, and the FileManager for saving and loading game data. Also worked on documentation.

3. Features

3.1 Mandatory Features

We implemented all the required features:

- OOP design — classes use encapsulation, inheritance, and polymorphism
- All code is in English
- Android app built in Java with Android Studio
- You can create crew members — Pilot, Engineer, Medic, Scientist, Soldier
- Training system — crew gets XP in the Simulator which increases their skill
- Turn-based mission system — crew fights threats, threat hits back each turn
- Energy system — crew restores full energy when they return to Quarters
- Uses HashMap and ArrayList for data storage

3.2 Bonus Features

RecyclerView (+1)

All crew lists use RecyclerView. The CrewMemberAdapter shows each crew member as a card with their name, stats, HP, and image.

Crew Images (+1)

Each specialization has its own drawable image file (crew_pilot.xml, crew_engineer.xml, etc.). The images are different colors and shapes so you can tell them apart easily.

Mission Visualization (+2)

During missions there is a live HP bar for the threat that looks like [▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓]. It updates after every action. The mission log also auto-scrolls to the bottom so you always see the latest line.

Tactical Combat — Attack, Defend, Special (+2)

The player picks an action every turn. There are 3 buttons:

- Attack — normal hit with some randomness
- Defend — reduces damage taken this turn
- Special — hits harder but costs 2 of your own HP

These are built using the MissionAction interface. AttackAction, DefendAction, and SpecialAction all implement it. This is polymorphism — the code calls action.execute() and Java picks the right one at runtime.

Statistics (+1)

There is a Statistics screen that shows how many missions each crew member has done, how many they won, and how many training sessions they attended. It also shows colony-wide totals like total missions and win rate.

No Death / MedBay (+1)

When a crew member loses all HP they don't get deleted. Instead they go to the MedBay. Their XP and HP get reset as a penalty. After 2 missions pass, you can discharge them back to Quarters.

Randomness in Missions (+1)

Both crew attacks and threat attacks have some randomness. The damage is multiplied by a random number between 0.8 and 1.2 so each fight feels a little different.

Specialization Bonuses (+2)

Each crew type gets +2 skill against a specific type of threat:

- Pilot — +2 vs navigation threats like Asteroid Storm and Meteor Shower
- Engineer — +2 vs technical threats like Hull Breach, Fuel Leak, Reactor Meltdown
- Medic — +2 vs medical threats like Oxygen Failure
- Scientist — +2 vs science threats like Solar Flare and Cosmic Radiation
- Soldier — +2 vs combat threats like Alien Attack and Rogue AI

This works because each subclass overrides act(Threat threat) and checks the threat category. This is also polymorphism.

Larger Squads — 3 Members (+2)

You can send 2 or 3 crew members on a mission. The turn system loops through all squad members and skips anyone who has been knocked out.

Fragments (+2)

The whole app is built with fragments inside one MainActivity. There is a bottom navigation bar with 5 tabs. We use 8 fragments in total: Home, Recruit, Quarters, MissionControl, Simulator, MedBay, Statistics, and More.

Data Save and Load (+2)

The game saves everything to a file called spacecolony_save.txt on the device. It saves each crew member's name, stats, XP, and location. When you open the app it loads the file automatically. You can also press SAVE COLONY and LOAD COLONY manually on the Home screen.

3.3 Bonus Features Summary

Feature	Points	Status
RecyclerView	+1	Done
Crew images	+1	Done
Mission visualization (HP bar + log)	+2	Done
Tactical combat (Attack / Defend / Special)	+2	Done
Statistics tracking	+1	Done
No death / MedBay	+1	Done
Randomness in missions	+1	Done
Specialization bonuses	+2	Done
Larger squads (up to 3 members)	+2	Done
Fragments UI	+2	Done
Data save and load	+2	Done

4. OOP Design

4.1 Encapsulation

All class fields are private. For example, in CrewMember, all 9 fields (id, name, specialization, skill, resilience, maxEnergy, energy, experience, idCounter) are private. You can only access them through getters and setters. Same for Storage, MedBay, and all other classes.

4.2 Inheritance

CrewMember is the base class. Pilot, Engineer, Medic, Scientist, and Soldier, all extend it. Each subclass just calls super() with different stat values. This avoids repeating the same code 5 times.

4.3 Polymorphism

We use polymorphism in two ways. First, the MissionAction interface is implemented by AttackAction, DefendAction, and SpecialAction. The app calls action.execute() and Java decides

at runtime which one to run. Second, each crew subclass overrides act (Threat threat) to add their bonus, and getInfo() to show a different prefix. The rest of the code doesn't need to know which subclass it is working with.

4.4 Packages

- model - CrewMember, 5 subclasses, Threat
- control - Quarters, Simulator, MissionControl
- mission - MissionAction interface, 3 action classes
- storage - Storage class
- data - GameData
- statistics - StatisticsManager, CrewStats
- medbay - MedBay
- filemanager - FileManager
- ui / ui.fragments - MainActivity and 8 fragments

5. How to Use the App

Step	Screen	What to do
1	Recruit	Enter a name, pick a specialization, press RECRUIT. The crew member goes to Quarters.
2	Quarters	Select crew and press Move to Simulator to train them.
3	Simulator	Select crew and press TRAIN. They get +1 XP. Send them back to Quarters when done.
4	Quarters	Select 2 or 3 crew and press Move to Mission Control.
5	Mission	Select the crew, press LAUNCH MISSION. A threat appears.
6	Mission	Press Attack, Defend, or Special each turn. The threat hits back automatically.
7	After mission	Survivors return to Quarters with +1 XP. Defeated crew go to MedBay.
8	MedBay	After 2 missions pass, you can discharge recovered crew back to Quarters.
9	Home	Press SAVE COLONY to save your progress to the device.

6. Tools We Used

- Android Studio - for writing and running the Java code
- Java - the programming language
- Lucidchart - to draw the UML diagram
- GitHub - to store and share the code , for research on similar projects
- Claude, Chatgpt - helped with code and documentation
- Gemini, ChatGPT - used during planning for brainstorming ideas

7. Disclaimer on AI Usage

Planning phase

As we mentioned in our project plan before during the planning phase, we used Gemini and ChatGPT to help organize our ideas. We used them to think through which classes we need, how they should connect, and which bonus features to include.

When we had errors and bugs

During development we ran into many errors. Whenever we got an error, we could not fix ourselves, we copied the error message and asked Claude or ChatGPT to help understand what was wrong and how to fix it. For example:

- The app was crashing because GameData was in the wrong package folder
- DefendAction was making the threat attack twice — Claude helped us find the double retaliation bug
- Statistics were being counted wrong for 3-person squads — the arguments were in the wrong order
- The mission log was not scrolling — Claude helped add the auto-scroll fix
- FileManager had import errors that needed to be cleaned up etc...

GitHub search

We also searched GitHub for similar Android OOP projects to get ideas on how other students structured their code. It helped us understand how to organize the packages and fragments better.

AI was also used for brainstorming, especially when deciding how to include bonus features like the MedBay system and tactical combat.

In addition, AI helped with wording and formatting parts of the documentation and was used throughout the development process to fix bugs, resolve errors, and improve the code.

All final decisions, design choices, and content were reviewed and adjusted by our team.

8. UML Class Diagram

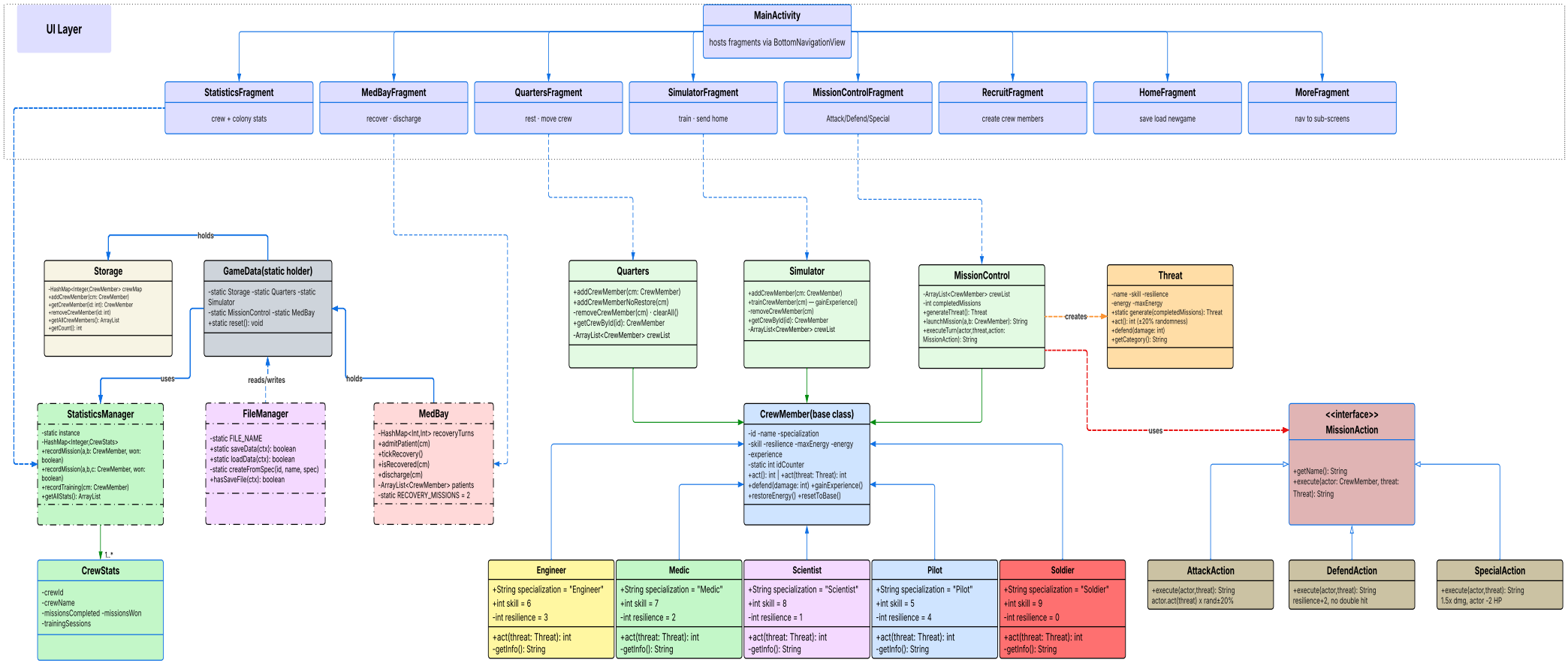
The UML diagram was updated and improved from our original project plan. The final version reflects all the changes we made during development, including new classes that were added such as MedBay, FileManager, StatisticsManager, CrewStats, and the MissionAction interface with its three implementations.

The UML diagram shows all the classes in the project and how they connect to each other. The UI layer is at the top. Below that are all the domain and control classes.

The UML diagram is submitted as a separate file: **UML_diagram_final.pdf**

Arrow types used in the diagram:

- Hollow triangle (solid line) — extends, used for the 5 crew subclasses → CrewMember
- Hollow triangle (dashed line) — implements, used for AttackAction / DefendAction / SpecialAction → MissionAction
- Dashed arrow — depends on or creates, used for fragments → controllers and MissionControl → Threat


UI Layer
MainActivity
hosts fragments via BottomNavigationView
StatisticsFragment
crew + colony stats
MedBayFragment
recover · discharge
QuartersFragment
rest · move crew
SimulatorFragment
train · send home
MissionControlFragment
Attack/Defend/Special
RecruitFragment
create crew members
HomeFragment
save load newgame
MoreFragment
nav to sub-screens
holds
Storage
-HashMap<Integer,CrewMember> crewMap
+addCrewMember(cm: CrewMember)
+getCrewMember(id: int): CrewMember
+removeCrewMember(id: int)
+getAllCrewMembers(): ArrayList
+getCount(): int
GameData(static holder)
-static Storage -static Quarters -static Simulator
-static MissionControl -static MedBay
+static reset(): void
Quarters
+addCrewMember(cm: CrewMember)
+addCrewMemberNoRestore(cm)
-removeCrewMember(cm) · clearAll()
+getCrewById(id): CrewMember
-ArrayList<CrewMember> crewList
Simulator
+addCrewMember(cm: CrewMember)
+trainCrewMember(cm) — gainExperience()
-removeCrewMember(cm)
+getCrewById(id): CrewMember
-ArrayList<CrewMember> crewList
MissionControl
-ArrayList<CrewMember> crewList
-int completedMissions
+generateThreat(): Threat
+launchMission(a,b: CrewMember): String
+executeTurn(actor,threat,action: MissionAction): String
creates
Threat
-name -skill -resilience
-energy -maxEnergy
+static generate(completedMissions): Threat
+act(): int (±20% randomness)
+defend(damage: int)
+getCategory(): String
uses
reads/writes
holds
StatisticsManager
-static Instance
-HashMap<Integer,CrewStats>
+recordMission(a,b: CrewMember, won: boolean)
+recordMission(a,b,c: CrewMember, won: boolean)
+recordTraining(cm: CrewMember)
+getAllStats(): ArrayList
FileManager
-static FILE_NAME
+static saveData(ctx): boolean
+static loadData(ctx): boolean
-static createFromSpec(id, name, spec)
+hasSaveFile(ctx): boolean
MedBay
-HashMap<Int,Int> recoveryTurns
+admitPatient(cm)
+tickRecovery()
+isRecovered(cm)
+discharge(cm)
-ArrayList<CrewMember> patients
-static RECOVERY_MISSIONS = 2
CrewMember(base class)
-id -name -specialization
-skill -resilience -maxEnergy -energy
-experience
-static int idCounter
+act(): int | +act(threat: Threat): int
+defend(damage: int) +gainExperience()
+restoreEnergy() +resetToBase()
uses
<<interface>>
MissionAction
+getName(): String
+execute(actor: CrewMember, threat: Threat): String
1..*
CrewStats
-crewId
-crewName
-missionsCompleted -missionsWon
-trainingSessions
Engineer
+String specialization = "Engineer"
+int skill = 6
-int resilience = 3
+act(threat: Threat): int
-getInfo(): String
Medic
+String specialization = "Medic"
+int skill = 7
-int resilience = 2
+act(threat: Threat): int
-getInfo(): String
Scientist
+String specialization = "Scientist"
+int skill = 8
-int resilience = 1
+act(threat: Threat): int
-getInfo(): String
Pilot
+String specialization = "Pilot"
+int skill = 5
-int resilience = 4
+act(threat: Threat): int
-getInfo(): String
Soldier
+String specialization = "Soldier"
+int skill = 9
-int resilience = 0
+act(threat: Threat): int
-getInfo(): String
AttackAction
+execute(actor,threat): String
actor.act(threat) x rand±20%
DefendAction
+execute(actor,threat): String
resilience+2, no double hit
SpecialAction
+execute(actor,threat): String
1.5x dmg, actor -2 HP